JobTracker Hub

Item 7 — Development Log & Checkpoint History

What this document is. A running log of the Item 7 (Application Timeline) feature work, written so a new chat session – with no memory of previous ones – can pick up exactly where the last one stopped. This is a working document, not the end-user manual (see `JobTracker_User_Guide.pdf` for that) and not the Item 6 log (see `ITEM6_DEV_LOG.tex` for that item's own history).

Feature	Item 7 — Application Timeline (document-derived events + status-history-backed current status)
Workflow	One chat = one tested, zipped checkpoint. Claude does not touch GitHub or build the DMG.
Base project	JobTracker Hub, Item 6 / Checkpoint 6 sealed, 144/144 tests passing before this log's Checkpoint 1.
Scope reference	`ITEM7_TIMELINE_FDD_DRAFT.md` at the repository root defines v1 scope in full; this log records what was actually built and verified against it.
This log	Updated at the end of every checkpoint chat; latest checkpoint's "Next Steps" box is always the first thing to read.

Repository: https://github.com/willmaddock/jobtracker-hub
Log version: Checkpoint 1 (sealed) | **Updated:** August 29, 2026

Hand this PDF (or its `.tex` source) plus the latest checkpoint zip to the next chat as its starting context.

Contents

1 How To Use This Document

For the human. At the start of a new chat, upload the latest checkpoint zip and this `.tex` (or the compiled PDF), plus `ITEM6_DEV_LOG.tex` for the preceding item's history. Tell the new Claude session to read the Next Steps box for the most recent checkpoint before doing anything else.

For the next Claude session. This log is cumulative – do not delete earlier checkpoint sections when you add a new one. Append a new `Checkpoint N` section below the most recent one, update the Next Steps box to reflect the new state, and bump the cover page's checkpoint number and date. Keep every checkpoint's original "What was done" section intact as history, even after its follow-up items are completed elsewhere.

1.1 Ground rules that apply to every checkpoint

- Claude works only on the local project source. No git commit, no git push, no GitHub Releases, no building the `.dmg`.
- The human integrates each checkpoint zip into their real working repository, handles GitHub, builds the DMG, and performs real Safari/packaged-app validation.
- Every checkpoint must leave the existing test suite passing. Existing tests are never weakened or removed to make new tests pass.
- A checkpoint is produced only when the working tree has reached a coherent, testable state – never a zip of broken work just because the chat is ending.
- Extraction and inference stay deterministic and local (regex/heuristics), never a required cloud/AI dependency, per the project's local-first privacy posture.
- **Keep this log from growing without bound.** Once more than 3 checkpoints exist, condense every checkpoint older than the most recent 3 down to a short paragraph (scope + one-line outcome + deliverable filename) instead of its full "What was done" detail. Never condense the *Next Steps* section, and never condense any of the 3 most recent checkpoints.

2 Next Steps

Read this first. This box always reflects the state after the *most recently completed* checkpoint. If you are starting a new chat, this is your starting instruction.

As of the Item 7 closeout, Item 7 is sealed. The recommended next task is:

1. **Scope Item 8 only – do not implement it in the same chat that scopes it.** The archive/lifecycle engine, deferred since Item 6's Checkpoint 1, is the standing candidate. No design work on it has begun.
2. If Item 8 is deferred further, the FDD draft's out-of-scope list (§3.1) is the place to start for a v2 Timeline scope: multi-transition status history on the Timeline itself (not just the latest), and/or a UI affordance for the identical-status-saved-twice case's manual verification gap noted in §3.5.
3. **Git has not been committed or pushed for either CP6 (Item 6) or Item 7.** Both are sealed and ready; committing (one or two commits, as previously decided), pushing, and building the next DMG are the user's own next steps against their controlled local repository.

Not yet started: everything in the items above. No Item 8 code, design document, or FDD exists yet.

3 Checkpoint 1 — Application Timeline (Document Events + Status History)

3.1 Scope

Build the Timeline view recommended at the end of Item 6: the shape of an application's life – applied, interviewed, resolved – derived from evidence the app already extracts, per `ITEM7_TIMELINE_FDD_DRAFT.md`. Full FDD text lives at the repository root; the v1-scope summary that matters for this checkpoint:

> **v1 scope, in brief.** Two kinds of Timeline entry: (1) **document-derived events**, one per `application_confirmation`/`interview_notice` document with a detected date, never merged even when a folder has more than one of either type; (2) a single item-level **"Current status"** entry, backed by a new append-only `status_history` log rather than a document, since the real corpus has almost no standalone rejection documents to derive a rejection event from. Explicitly out of v1 scope: recovering a transition date for any status set before `status_history` existed, and showing more than the single latest transition to the current status.

3.2 What was done

- `_app/dossier.py`: `assemble_dossier()` now also returns `timeline_events` – a list built from the new `TIMELINE_EVENT_DOC_TYPES = ("application_confirmation", "interview_notice")` tuple, reusing the exact same per-document `detected_date_applied` that `extract.py` already computes for every document regardless of type (no new extraction logic). Every matching document becomes its own event; a folder with a phone-screen request and a later interview request produces two entries, not one. Events sort by `(date, doc-type-order, relpath)`, with `application_confirmation` breaking a same-date tie before `interview_notice` via `_TIMELINE_TYPE_ORDER`.
- `_app/overrides_store.py`: new append-only `status_history` table (`id`, `item_key`, `status` – the resulting *effective* status, so a reset-to-auto is still a real findable transition – `changed_at`, `source`), plus an index on `item_key`. New functions `append_status_history` (no-op on an exact repeat of the current status – no duplicate row), `get_status_history`, `get_latest_status_change` (the most recent row matching a given status, correctly returning the *second* occurrence after a re-open), and `delete_status_history` (called wherever an item's overrides are deleted, so removed applications don't leave orphaned history).
- `_app/api.py`: `save_override()` and `bulk_override()` both call `append_status_history()` whenever the effective status actually changes. `/api/applications/\{item\_id\}/dossier` adds `current_status`, `current_status_date` (`YYYY-MM-DD` from `get_latest_status_change`, or `None`), and `current_status_date_known` (`False` when no matching history row exists – e.g. a status set before this table shipped – so the UI reports the date as genuinely unknown rather than guessing). Override-deletion paths also call `delete_status_history`.
- **Tests:** `tests/test_timeline.py` (new, 7 tests) – confirmation-only events, interview-only events, multiple interview documents each producing their own event, chronological cross-doc-type sort, same-date tiebreak ordering, an empty timeline when no dated evidence exists, and `timeline_events` coexisting correctly alongside Checkpoint 6's `date_applied` auto-fill in the same response. `tests/test_status_history.py` (new, 14 tests) – direct unit tests of `overrides_store.py`'s new functions (table creation, append/get, no-duplicate-on-repeat-save, multi-step transition sequencing, latest-match lookup including the re-opened-status case, no-history case) plus end-to-end API tests covering `save_override`/`bulk_override` logging and `current_status_date_known`'s honest-unknown behavior for a pre-existing status.

Design note – why status history, not a document-derived rejection event. The FDD draft's originally discussed six-stage abstract event model included a document-derived rejection stage. Real-corpus review during implementation showed this doesn't match the data: rejection is very rarely communicated as its own distinct document in the working tracker, if at all, compared to confirmation and interview-request emails. Rather than build detection logic against evidence that mostly doesn't exist, rejection (and any other status) surfaces through the same item-level `status_history` log that already exists for `manual_status` tracking – the "Current status" Timeline entry, not a fabricated document event.

Design note – append-only, no-op on repeat, by design. `status_history` exists to answer "when did this become X", not to log every save click. Two consecutive saves of the same effective status (e.g. re-saving a note without changing the status dropdown) must not insert a second row, or `get_latest_status_change` would report the second click's timestamp as the transition date even though nothing actually changed. `test_repeated_save_of_same_status_does_not_duplicate_history` exercises this directly.

3.3 Test results

```
$ python3 -m pytest -q
144 passed, 1 warning
```

All 123 pre-existing tests pass unchanged, plus the 21 new Item 7 tests above (7 in `test_timeline.py`, 14 in `test_status_history.py`). Run for real against the FastAPI `TestClient` against the working environment – this checkpoint's implementation session, like Checkpoints 3 and 6 of Item 6 before the real-app follow-up, is recorded here as reported from that run rather than re-executed by every later chat that reads this log, since this closeout session's own sandbox has no network access to install `pytest`/`fastapi`/`httpx`.

3.4 Post-checkpoint verification: manual click-through against the real packaged app

A manual click-through was carried out in a later session directly against the real working tracker (`working-db.zip`), using both the local dev server (Safari, `127.0.0.1`) and the packaged `JobTracker Hub.app` DMG build. **Two separate, out-of-sync data copies were involved at different points** – see the note below before reading the results.

- **Confirmation-only and confirmation+interview timeline events rendered correctly**, each showing the expected date and source document.
- **Multiple-interview handling validated**: a folder with more than one interview-related document produced a separate Timeline entry for each, not a single merged line.
- **A pre-existing rejection (set before `status_history` shipped) correctly showed its status date as unknown** rather than a fabricated or backfilled date – confirming `current_status_date_known: false` behaves as designed against real, not synthetic, pre-existing data.
- **Restart durability, proven twice.** An application's status (Apple Retail) was set, saved, and the packaged app was then fully quit and relaunched from the DMG – `Current status: Applied - since 2026-08-29` was still shown afterward, read from `overrides.db` on disk rather than anything held in memory. This was confirmed a second time later in the same session after an additional import.
- **A real multi-step transition round trip.** Apple Retail was moved Applied → Interviewing → Applied in the packaged app, with a "Saved." toast and correct pipeline-count movement (41→40 applied / 13→14 interviewing, then back) on each of the two saves.

- `date_applied` **stayed independent of Timeline events** throughout – no cross-talk observed between the Checkpoint-6 date-applied auto-fill/conflict/provenance behavior and the new Timeline entries.

Important: two application instances were open during validation. A Safari window against the local dev server (`127.0.0.1`) and the packaged app were tracking two different, out-of-sync copies of the underlying data at different points in the session – at one point an imported `working-db.zip` snapshot predated an edit made in the other window, meaning that edit (a Brother status change) landed in a workspace copy that was no longer the active one afterward, not a data-loss bug. The dev-server window's own stale-display behavior (e.g. a Timeline line not refreshing immediately after a save made in a different window entirely) reflects that separate workspace and is **not** recorded as an Item 7 defect. Only the packaged application's results, listed above, count as the official Item 7 real-app validation.

3.5 Known limitations

- **The identical-status-saved-twice case was not independently isolated as its own step in the manual click-through.** The real click-through covered two genuine transitions (Applied → Interviewing → Applied), not a same-status-twice repeat save specifically. The no-duplicate-row behavior itself is fully covered at the unit level (§3.2, `test_repeated_save_of_same_status_does_not_du` part of the passing 144/144 run) – this is a gap in independent real-app corroboration of an already-verified case, not an open correctness question.
- Cannot recover a transition date for any status set before `status_history` existed, by design – see the FDD draft and `current_status_date_known`.
- The Timeline's "Current status" entry shows only the single latest transition to the current status, not a full multi-transition history, in v1.
- Carries forward all Item 6 (Checkpoints 1–6) limitations unchanged (US-format phone regex, empty-password-only PDF decryption, no `.docx` support, trailing-boilerplate bleed into the last-matched role section, arbitrary-but-deterministic alphabetical job-posting tiebreak, no per-document source attribution on merged contacts, English-month/US-slash-date detection only, no signal for which confirmation document wins when a folder has more than one).

3.6 Deliverable

`jobtracker-hub-item7-merged.zip` (implementation) and this closeout's handoff zip – full repository source (minus `.venv`, `.git`, caches, and local workspace state), plus this development log, `ITEM7_TIMELINE_FDD_DRAFT.md`, and an updated `CHANGES.md` at the repository root.

4 Item 7 Closeout

Item 7 is sealed. Automated tests: 144/144 (21 of them new to Item 7). Real packaged-app validation: pass, per §3.4, with the Safari/dev-server discrepancy explicitly excluded from the result per the warning box above. Checkpoint 6 of Item 6 remains sealed and was confirmed unaffected. No Item 8 work – design or implementation – has begun. Git has not been committed or pushed; that, along with building the next DMG, remains the user's own next step against their controlled local repository, per the workflow described in §1.